

NVIDIA AI/ML Infrastructure – Study Review

Contents

1	Mental Model of AI/ML Infrastructure	3
1.1	Hardware	3
1.2	GPU Architecture	4
1.3	Network	4
1.4	Storage	5
1.5	Scheduler	5
1.6	Runtime	6
1.7	Framework	6
1.8	Model	7
1.9	Experiment Tracking	7
2	PyTorch Distributed Components by Responsibility	8
2.1	Explain End-to-End Training Flow	9
3	Distributed Training – FULL DEPTH	9
3.1	DDP Internals	9
3.2	FSDP / ZeRO – Memory Deep Dive	15
4	Parallelism Strategies for LLMs	17
4.1	Data Parallelism	17
4.2	Tensor Parallelism	17
4.3	Pipeline Parallelism	18
4.4	Hybrid Parallelism	19
5	NCCL + Networking	20
5.1	AllReduce Algorithms	20
5.2	InfiniBand + RDMA	21

6 GPU Architecture + Performance	23
6.1 SM Architecture	23
6.2 Bottleneck Types	24
7 Data Pipeline Scaling	25
8 Fault Tolerance + Checkpointing	27
8.1 Checkpoint and Resume State	28
9 Mixed Precision + Numerical Stability	30
9.1 FP16 vs BF16	30
9.2 Loss Scaling	30
9.3 Gradient Overflow	30
9.4 Stability Tradeoffs	31
10 Scheduler + Orchestration	31
10.1 Slurm	31
10.2 Kubernetes	32
11 Storage Systems	34
12 Cloud vs On-Prem Tradeoffs	36
13 Researcher Productivity Metrics	40
14 Rapid-Fire Questions Section	42
15 Whiteboard System Design Prompts	43
15.1 Design multi-tenant GPU scheduling system	44
15.2 Design distributed LLM inference cluster	44
15.3 Design hyperparameter search system	45
15.4 Design fault-tolerant distributed training	46
15.5 Design GPU utilization monitoring dashboard	46
15.6 Multi tentnet gpu for heavey tranformer model	47
16 Your Personal Deep Dive Stories	47
16.1 WaveHPC Impact Story	47
16.2 High-Performance Matrix Inversion via Custom GPU Kernels	48
16.3 GPU-Accelerated Image Processing Pipeline	49
16.4 Convolutional Layer Optimization with Triton	50

1 Mental Model of AI/ML Infrastructure

Layered Stack

Hardware → GPU Architecture → Network → Storage → Scheduler → Runtime → Framework → Model → Experiment Tracking

Section 1 at a Glance (Recall Outline)

1. **Hardware:** How cluster and node topology is wired
2. **GPU Architecture:** SM, warp, memory hierarchy, precision units
3. **Network:** Ethernet vs InfiniBand, RDMA/GPUDirect, management planes
4. **Storage:** Fast-to-slow hierarchy and I/O bottleneck signs
5. **Scheduler:** Slurm/Kubernetes resource orchestration
6. **Runtime/Framework:** CUDA + NCCL + PyTorch distributed stack
7. **Model/Tracking:** Model shape drives infra; tracking validates impact

One-Line Exam Mental Model

If GPUs are idle, find the starvation layer: input (**storage/data**), communication (**network/NCCL**), or runtime/configuration (**scheduler/framework**).

1.1 Hardware

Hardware Recall

Path: User → Login Node → Compute Node(s) ↔ Shared Storage

HPC stack view (cluster-level):

- User connects to a login/head node.
- Login/head nodes connect to compute nodes.
- Compute nodes share a parallel file system and are interconnected.
- Storage systems are typically separate from compute clusters.

Inside a compute node:

- Main components: CPU, GPU, and (in many modern systems) DPU/SmartNIC.
- CPU and GPU are connected through PCIe.
- GPUs within a node are connected through NVLink or NVSwitch.

1.2 GPU Architecture

GPU Recall

Think in levels: GPU $\rightarrow$ SM $\rightarrow$ Warp (32 threads) $\rightarrow$ Execution units + memory hierarchy.

- Each GPU has multiple **SMs** (Streaming Multiprocessors).
- SMs share global memory (HBM/DRAM) through shared L2.
- Inside each SM: register files, warp scheduler, dispatch logic, L1/shared memory, and execution cores.
- A warp is typically 32 threads, and schedulers pick ready warps to improve occupancy and hide latency.
- Cores may target different precisions: Tensor Core ops, FP32, INT, etc.

1.3 Network

Bottleneck Clue

If network or storage is busy while GPUs are idle, training is often stalled by gradient synchronization or data movement.

Network Recall

Data-plane speed tools: InfiniBand + RDMA (+ GPUDirect where supported). **Control/ops planes:** in-band for normal ops, out-of-band for hardware recovery/management.

- Ethernet is commonly used for general cluster traffic.
- InfiniBand is commonly used as the high-speed training interconnect.
- RDMA enables low-overhead data movement with minimal CPU involvement.
- GPUDirect RDMA/GPUDirect Storage can reduce unnecessary host copies in supported setups.
- DMA: direct memory transfer between device and host memory without active CPU copying loops.
- In-band network: regular OS/service traffic (SSH, scheduler, etc.).
- Out-of-band management network: hardware management path even if host OS is down.

1.4 Storage

Bottleneck Clue

If DataLoader is maxed out and GPUs are idle, storage/input pipeline is likely the bottleneck.

Storage Recall

Order (fast to slow): NVMe → Parallel FS → NFS → Object Store.

Fastest to slowest (practical training view):

1. **Local NVMe:** fastest scratch storage for temporary datasets/intermediate files.
2. **Parallel file systems** (Lustre, GPFS, BeeGFS): shared high-throughput storage across nodes.
3. **NFS:** good for configs/scripts/small files, not ideal for heavy training I/O.
4. **Object storage** (S3-style): good for logs/checkpoints/archive, typically higher latency for hot training reads.

1.5 Scheduler

Common options: Slurm or Kubernetes.

Scheduler Recall

Slurm: batch/HPC-first job scheduling. **Kubernetes:** container-first orchestration with GPU plugins.

Area	Slurm (high-level)	Kubernetes (high-level)
Workload model	Batch/HPC scheduler for queued jobs and resource allocation.	Container orchestrator for pod-based workloads.
GPU resource model	GPU resources configured via <code>slurm.conf</code> and <code>gres.conf</code> .	GPUs requested through Kubernetes resource specs and exposed via device plugins (e.g., NVIDIA plugin).
Scheduling fairness	Priorities, QoS, fairshare, and preemption policies.	Scheduler places pods on nodes with available GPUs; policies are cluster-config driven.
Monitoring ops	<code>sacct</code> , <code>sreport</code> , dashboards, and node/GPU health logs.	Strong observability ecosystem (e.g., Prometheus/Grafana), plus autoscaling support.
Performance capacity	Tune GPU placement, NCCL config, CPU/GPU affinity, and network settings; plan capacity from workload trends.	Optimize placement and scaling behavior through scheduling constraints and autoscaling signals.

Slurm example command:

```
srun -partition=gpu -gres=gpu:1 -mem=16G
      -time=01:00:00 -pty bash
```

- `srun`: starts an interactive run.
- `-gres=gpu:1`: requests one GPU.
- `-mem=16G -time=01:00:00`: sets memory and time limits.
- `-pty bash`: opens an interactive shell on the allocated node.

1.6 Runtime

Runtime Recall

Runtime stack: CUDA kernels + NCCL collectives + drivers + container environment.

- CUDA runtime and kernels (compiled with `nvcc`).
- NCCL for distributed GPU communication.
- Containers for reproducibility.
- Drivers as the host-to-GPU interface.

Common issues: driver/runtime mismatch, inefficient kernel launches, communication back-end bottlenecks, or container misconfiguration.

1.7 Framework

PyTorch

Framework Recall

DDP: replicate model, sync gradients. **FSDP:** shard model states to fit larger models.
TP/PP: split model compute across dimensions/stages.

DDP (*DistributedDataParallel*):

- One model replica per GPU; gradients synchronized via AllReduce.
- Typical setup: wrap model with DDP, use `DistributedSampler`, call `loss.backward()` normally.
- Scaling can suffer with very small per-GPU batches because communication dominates.

FSDP (*Fully Sharded Data Parallel*):

- Shards parameters, gradients, and optimizer states across ranks.
- Useful for fitting larger models.
- Combine with mixed precision/checkpointing for memory efficiency.

Tensor and pipeline parallelism:

- Key when model size exceeds what plain data parallelism can handle.
- Directly affects communication pattern, memory pressure, and scaling behavior.

1.8 Model

Model Recall

Model architecture determines memory pressure, communication pattern, and best parallelism strategy.

Model size, structure, and compute pattern heavily influence infrastructure decisions: parallelism strategy, memory requirements, communication overhead, and inference latency.

1.9 Experiment Tracking

Tracking Recall

No tracking means no reliable diagnosis, no reproducible comparisons, and weak optimization decisions.

Tools like Weights & Biases and monitoring dashboards provide visibility into training performance and regressions. Without tracking, it is difficult to diagnose failures, evaluate optimizations, or scale workflows reliably.

2 PyTorch Distributed Components by Responsibility

<i>Responsibility</i>	<i>Component</i>	<i>What It Does</i>	<i>Implementation Steps</i>
Multi-process launching	Torchrun	Spawns one process per GPU and sets rank/world size	Launch with <code>torchrun -nproc_per_node=N</code> ; read <code>LOCAL_RANK</code> ; set device; init process group.
Process group init	<code>init_process_group</code>	Creates communication group (NCCL/Gloo)	Call <code>torch.distributed.init_process_group(...)</code> early with <code>backend="nccl"</code> and set local CUDA device.
Distributed data parallel	DDP	Replicates model on each GPU and syncs gradients	Wrap model in DDP, use <code>DistributedSampler</code> , call <code>loss.backward()</code> .
Distributed sampling	<code>DistributedSampler</code>	Shards dataset across ranks	Pass sampler into <code>DataLoader</code> and call <code>sampler.set_epoch(epoch)</code> .
Fully sharded data parallel	FSDP	Shards parameters, gradients, and optimizer state	Use <code>torch.distributed.fsdp.FullyShardedDataParallel</code> and configure sharding strategy, mixed precision, and wrapping policy.
Sharded checkpointing	<code>dist.checkpoint</code>	Saves/restores large distributed states efficiently	Use <code>torch.distributed.checkpoint</code> with FSDP-compatible state-dict APIs for save/load.
Gradient accumulation	<code>no_sync()</code> (DDP)	Avoids sync on intermediate microsteps	Use <code>with model.no_sync():</code> and sync on final accumulation step.
Mixed precision	<code>autocast</code> + <code>GradScaler</code>	Speed + memory improvements with lower precision	Wrap forward pass in <code>autocast()</code> , scale gradients with <code>GradScaler</code> .
Activation checkpointing	checkpoint API	Trades compute for lower memory usage	Use <code>torch.utils.checkpoint</code> on large blocks/layers and recompute on backward.
Tensor parallelism	<code>device_mesh</code> + <code>DTensor</code>	Splits tensors/layers across GPUs	Define mesh, shard tensors, coordinate collectives.
Pipeline parallelism	Pipeline APIs/ecosystem tools	Splits model into stages via microbatches	Partition model stages and tune schedule to reduce bubbles.
Communication backend	NCCL	Handles GPU collectives	Use <code>backend="nccl"</code> and tune/debug via environment settings.
Compilation / optimization	<code>torch.compile</code>	Graph capture and kernel fusion	Compile model and validate with profiler-based speedups.
Data loading optimization	<code>DataLoader</code> tuning	Keeps GPUs fed	Tune <code>num_workers</code> , <code>pin_memory</code> , prefetch settings.
Profiling	<code>torch.profiler</code>	Captures CPU/GPU/comm timelines	Run scheduled traces and inspect bottlenecks.
Fault tolerance / elastic	TorchElastic (<code>torchrun</code>)	Supports restarts and membership changes	Use rendezvous backend + robust checkpoint resume flow.

2.1 Explain End-to-End Training Flow

Whiteboard explanation from data loading to optimizer step across 32 GPUs.

Likely Question

Walk me through how a distributed PyTorch job runs across multiple nodes.

Typical flow:

- Start with a launcher (`torchrun`, often orchestrated by Slurm/Kubernetes).
- Spawn one process per GPU and assign each process a global rank, local rank, and world size.
- Each process sets its CUDA device and initializes the distributed process group (usually NCCL).
- Model is wrapped in DDP (or FSDP for sharded training).
- Dataset is partitioned with `DistributedSampler` so each rank gets a different data shard.
- During training, each rank runs forward/backward locally.
- Gradient synchronization (AllReduce or sharded collectives) keeps model updates consistent.
- Optimizer step proceeds in lockstep across ranks.

Overall performance depends on compute, network bandwidth, storage throughput, data loader efficiency, and scheduler configuration. A bottleneck in any layer can reduce scaling efficiency.

3 Distributed Training – FULL DEPTH

3.1 DDP Internals

A mini-batch is a small subset of training examples processed in one forward + backward pass.

DDP Mental Model (Forward to Optimizer Step)

1. Forward pass runs independently on each GPU replica.
2. Backward pass computes parameter gradients.
3. Autograd hooks trigger communication when gradient buckets are ready.
4. NCCL AllReduce synchronizes gradients across ranks.

5. Optimizer step updates weights in lockstep across GPUs.

What Triggers Communication in DDP

- DDP registers autograd hooks on parameters.
- Hooks are attached to gradient accumulation logic.
- As gradients are produced in backward, they are added to buckets.
- When a bucket is ready, DDP schedules NCCL collective communication.

Why Overlap Matters

Communication starts during backward (not only after backward ends). This overlap of backward compute + NCCL communication reduces GPU idle time and hides communication latency.

Straggler effect (preserved + organized):

- Faster GPUs wait for slower ones at synchronization points.
- Typical causes: data imbalance, CPU data-loading bottlenecks, storage I/O stalls, or network congestion.
- Quick data check: compare local batch sizes/runtimes across ranks (for example, print-/trace per-rank batch stats).

Debug heuristics (preserved + corrected):

- **Data imbalance:** one rank receives more work; that GPU is idle *before* forward starts on next step.
- **CPU bottleneck:** DataLoader workers too few; CPU saturated (check with `htop`).
- **Disk/storage bottleneck:** slow reads from storage or network filesystem (check with `iostat` and storage metrics).
- **Network bottleneck:** GPUs wait during gradient sync, usually *after* backward kernels (check NCCL traces/`nsys`; use network tools such as `ifstat`/fabric metrics).
- **GPU utilization clues:** `nvidia-smi` shows low utilization or long idle gaps on specific ranks.

Topics to master:

- Autograd hooks
- Gradient bucketing: AllReduce starts when a gradient bucket is full, which enables overlap with backward compute.

- Bucket-size tradeoff: smaller buckets improve overlap but increase communication-call overhead; larger buckets reduce call count but can delay communication start.
- Tuning bucket size can improve scaling by balancing overlap vs communication overhead.
- Overlapping compute + communication
- NCCL backend: optimized for GPU collectives, uses NVLink/PCIe topology awareness, and builds efficient communication rings/trees.
- Rank initialization

Follow-Up They Might Ask

Why does small batch size hurt DDP scaling?

- With very small per-GPU batch size, compute per step is short while communication cost stays similar, so communication dominates.
- Increasing global batch size often requires learning-rate retuning (and sometimes warmup adjustments) for stable convergence.

At the end, overlap means backward compute continues while previous gradients are being reduced.

How to detect poor overlap:

- Large communication-only gaps after backward.
- High GPU idle time near the step tail.
- In profiler timelines (e.g., Nsight Systems), good runs show interleaving of backward kernels and NCCL ops; poor runs show big NCCL blocks after backward completion.

DDP Init and Failure Checklist

Each DDP process has a unique RANK; WORLD_SIZE is total processes.

Common env vars:

- MASTER_ADDR
- MASTER_PORT
- RANK
- WORLD_SIZE

If setup is wrong, typical symptoms:

- rank mismatch or world-size mismatch → hang/deadlock at init/collectives
- wrong master address/port conflict/firewall block → rendezvous failure
- Slurm misconfiguration → inconsistent rank mapping

DDP Step-by-Step (Interview Version)

Each GPU runs a full model replica on a different data shard. During backward, autograd hooks bucket gradients and trigger NCCL AllReduce. After synchronization, each rank applies the same optimizer step. **Key point:** gradient sync is overlapped during backward, not only after backward ends.

PyTorch FSDP: What Changes vs Vanilla Training

- launch one process per GPU (`torchrun` or Slurm launcher)
- initialize distributed process group (`init_process_group`)
- set local CUDA device per rank
- wrap model/submodules with FSDP
- train with normal forward/backward/step loop
- use FSDP-aware sharded checkpointing

Key APIs to know:

- `torchrun -nproc_per_node=...`
- `torch.distributed.init_process_group`
- `torch.distributed.fsdp.FullyShardedDataParallel`
- `torch.distributed.fsdp.wrap` (auto-wrap policy)
- `ShardingStrategy.FULL_SHARD` (ZeRO-3-like behavior)
- `FSDP.state_dict_type(..., SHARDED_STATE_DICT)`

Listing 1: FSDP pseudocode (PyTorch)

```
1 import os
2 import torch
3 import torch.distributed as dist
4 from torch.optim import AdamW
5 from torch.utils.data import DataLoader, DistributedSampler
6 from torch.distributed.fsdp import FullyShardedDataParallel as
   FSDP
7 from torch.distributed.fsdp import ShardingStrategy, StateDictType
8 from torch.distributed.fsdp.api import MixedPrecision
9 from torch.distributed.fsdp.wrap import
   size_based_auto_wrap_policy
10
11 def setup_dist():
```

```

12     dist.init_process_group(backend="nccl")
13     local_rank = int(os.environ["LOCAL_RANK"])
14     torch.cuda.set_device(local_rank)
15     return local_rank, dist.get_rank(), dist.get_world_size()
16
17 def wrap_with_fsdp(model):
18     auto_wrap = lambda m, recurse, nonwrapped_numel:
19         size_based_auto_wrap_policy(
20             m, recurse=recurse, nonwrapped_numel=nonwrapped_numel,
21             min_num_params=1e6
22         )
23     mp = MixedPrecision(
24         param_dtype=torch.bfloat16,
25         reduce_dtype=torch.bfloat16,
26         buffer_dtype=torch.bfloat16,
27     )
28     return FSDP(
29         model,
30         auto_wrap_policy=auto_wrap,
31         sharding_strategy=ShardingStrategy.FULL_SHARD,
32         mixed_precision=mp,
33         device_id=torch.cuda.current_device(),
34     )

```

FSDP FULL_SHARD Recall

FULL_SHARD is ZeRO-3-like: parameters, gradients, and optimizer states are sharded. Parameters are gathered just-in-time per wrapped unit; gradients are reduce-scattered in backward.

DeepSpeed ZeRO: What Changes vs Vanilla Training

- define DeepSpeed config with `zero_optimization.stage`
- call `deepspeed.initialize(...)`
- train with `engine.backward(loss)` and `engine.step()`
- save ZeRO-aware checkpoints via `engine.save_checkpoint(...)`

Listing 2: DeepSpeed ZeRO pseudocode

```

1 import deepspeed
2 import torch
3 from torch.utils.data import DataLoader, DistributedSampler
4

```

```

5 def get_ds_config(stage: int):
6     return {
7         "train_batch_size": GLOBAL_BS,
8         "train_micro_batch_size_per_gpu": PER_GPU_BS,
9         "gradient_accumulation_steps": GRAD_ACCUM,
10        "bf16": {"enabled": True},
11        "zero_optimization": {"stage": stage},
12        "optimizer": {
13            "type": "AdamW",
14            "params": {"lr": LR, "betas": [0.9, 0.999], "eps": 1e
15                        -8, "weight_decay": WD}
16        },
17    }

```

Quick Comms Notes

Ring AllReduce: gradients are split and passed around a ring; each rank talks to neighbors only, which is bandwidth-efficient. **Gradient accumulation:** accumulate multiple micro-batches before optimizer step to reduce sync frequency.

Scaling/Debug Interview Notes

Why efficiency drops at high GPU count: communication/synchronization dominates compute, latency and stragglers hurt more. **Common causes:** data-loader bottlenecks, too-small per-GPU batch, communication blocking, I/O saturation, CPU under-provisioning.

Job hang checklist:

- rank/world-size mismatch
- one crashed rank
- NCCL deadlock or collective-order mismatch
- firewall/port/master-address issues
- Slurm rank mapping errors

Single-node vs multi-node:

- single node (NVLink): higher bandwidth, lower latency, fewer failure points
- multi-node (InfiniBand/Ethernet): higher latency, more sync overhead, topology matters

3.2 FSDP / ZeRO – Memory Deep Dive

Baseline Memory in Vanilla DDP

Each GPU stores full parameters + full gradients + full optimizer states. Per-GPU memory scales poorly for very large models because model states are fully replicated.

ZeRO stands for *Zero Redundancy Optimizer*. It removes redundant model-state copies by sharding them across GPUs. FSDP is PyTorch’s ZeRO-style implementation.

Understand:

- **Parameter sharding:** each GPU stores only $1/N$ parameters at rest; parameters are gathered just in time per layer.
- **Gradient partitioning:** gradients are partitioned (typically via ReduceScatter), so each rank keeps only its shard.
- **Optimizer state partitioning:** major memory win; for Adam, optimizer state is often the largest memory component.
- **Activation checkpointing:** save fewer activations and recompute in backward (less memory, more compute).
- **Communication amplification:** ZeRO-3/FSDP reduces memory but increases communication events (gather/scatter).

Prompt

Explain tradeoffs between ZeRO-2 and ZeRO-3.

ZeRO-2

- shards optimizer states and gradients
- parameters remain replicated
- lower communication overhead than ZeRO-3
- easier to scale when network bandwidth is limited

ZeRO-3

- shards parameters + gradients + optimizer states
- maximum memory savings (roughly model-state memory $\sim 1/N$ per GPU)
- higher communication and more synchronization points
- enables larger model sizes at the cost of communication complexity

Memory sketch:

- DDP: params + grads + optimizer

- ZeRO-2: $\text{params} + \frac{\text{grads}}{N} + \frac{\text{optimizer}}{N}$
- ZeRO-3: $\frac{\text{params}}{N} + \frac{\text{grads}}{N} + \frac{\text{optimizer}}{N}$

When are parameters gathered in FSDP?

- just before a wrapped layer runs forward
- can be freed after layer compute
- gathered layer-by-layer (not entire model at once)

ReduceScatter vs AllReduce

- AllReduce: all ranks end with full reduced gradients
- ReduceScatter: reduce + shard in one step; each rank keeps only its shard

Memory-Spike Debug Guide (FSDP/ZeRO)

Where to inspect: forward gather, backward reduce-scatter, checkpointing, accumulation windows. Use `torch.cuda.memory_summary()`, Nsight Systems, and PyTorch profiler.

Common spike causes:

- coarse wrapping granularity (gathers too much at once)
- forward prefetch overlaps too many large gathers
- consolidated checkpoints gathering full model on rank 0
- mixed-precision misconfiguration creating unexpected FP32 copies
- large gradient-accumulation factor increasing peak live tensors

Checkpointing note:

- sharded checkpoints are most scalable for ZeRO/FSDP
- full consolidated checkpoints are easier to consume but can cause rank-0 memory spikes

4 Parallelism Strategies for LLMs

Memory Reminder

What usually blows up memory:

- steady-state memory: parameters, gradients, optimizer states
- peak memory: activations saved for backward (often grows with batch size and sequence length)

4.1 Data Parallelism

- Input batch is split across devices/processes.
- Each device runs forward + backward on its mini-batch independently.
- Pro: simple scaling, no pipeline bubbles, independent execution.
- Con: each device stores a full copy of model states (parameters + gradients + optimizer), so memory footprint is not reduced.
- Gradient synchronization happens through collectives (commonly AllReduce or ReduceScatter + AllGather, depending on implementation).

4.2 Tensor Parallelism

Tensor Parallelism (Intra-Layer)

Split tensors within a layer (for example, sharded matmul weights). This reduces per-device compute and per-device parameter footprint for that layer, but increases inter-device communication.

Row-wise vs column-wise split:

- **Column-parallel linear (split output features):** each GPU computes a slice of output; next layers may need AllGather (or compatible sharding). Backward usually includes gradient communication.
- **Row-parallel linear (split input features):** each GPU consumes input slices and partial outputs are summed (commonly via AllReduce).
- Frameworks place AllGather/AllReduce where cost is lowest.
- **Splitting weight matrices**
- Heavy Transformer ops affected: attention projections (Q/K/V/output) and FFN linear layers.

- Example (column split): for $W \in \mathbb{R}^{d \times 4d}$, shard columns across GPUs (e.g., GPU0: $W[:, 0 : 2d]$, GPU1: $W[:, 2d : 4d]$); each GPU computes local XW_i , then combine as needed.
- ***AllGather requirements***
- Partial outputs often must be gathered before the next layer.
- Gradients are reduced in backward (common patterns: AllGather, ReduceScatter, AllReduce).
- This communication occurs repeatedly across layers.
- ***Communication cost***
- Tensor parallelism introduces per-layer communication across TP groups.
- Communication cost scales with both layer count and TP size.

4.3 Pipeline Parallelism

Pipeline Parallelism

Layer-wise model split across devices. Memory is reduced per device because each stage stores only its own layers, but scheduling is required to minimize idle time (pipeline bubbles).

- Microbatching
- Without microbatching, devices idle heavily across stages.
- With microbatching, the pipeline fills:
 - Time 1: GPU0 $\rightarrow$ MB1
 - Time 2: GPU0 $\rightarrow$ MB2, GPU1 $\rightarrow$ MB1
 - Time 3: GPU0 $\rightarrow$ MB3, GPU1 $\rightarrow$ MB2, GPU2 $\rightarrow$ MB1
- Bubble overhead
- Idle time at start/end is the pipeline bubble.
- Bubble cost reminder: $(\text{\#stages} - 1) / \text{\#microbatches}$.
- More microbatches generally means a smaller bubble.
- Stage balancing
- If one stage is heavier, the whole pipeline stalls.
- Stages should be balanced for both compute and memory.

4.4 Hybrid Parallelism

Explain how large models combine DP (across replicas) + TP (within nodes) + PP (across layers).

Design Prompt

Design distributed training for a 100B parameter model on 64 GPUs. ***Placeholder retained:*** significance of this exact number to be filled later.

Why combine DP + TP + PP:

- TP helps when a single layer is too large for one GPU.
- PP helps when the full model is too large for one GPU.
- DP scales throughput across replica groups.

Communication differences:

- TP: heavy per-layer communication.
- PP: activation passing between stages.
- DP: gradient synchronization.
- Each type introduces a different bottleneck.

Questions to expect:

- How would you train a 100B model across 64 GPUs?
- What limits tensor-parallel scaling?
- What limits pipeline scaling?
- When do you prefer TP over PP?
- When do you prefer PP over TP?

Answer cues from your notes:

- combine TP (split large layers), PP (partition layers across stages), and DP (scale throughput across groups)
- TP limits: per-layer communication, interconnect bandwidth, latency
- PP limits: bubble overhead, stage imbalance, activation memory
- prefer TP when layers are extremely large and fast interconnect (e.g., NVLink) is available

- prefer PP when the full model does not fit a single GPU and cross-node bandwidth constraints dominate

5 NCCL + Networking

NCCL Recall

NCCL provides distributed collective communication primitives for multi-GPU training.

Core collectives from your notes:

- **Broadcast:** one rank sends data to all ranks (often weights/config).
- **AllReduce:** reduce across all ranks and store result on every rank (gradient sync).
- **Reduce:** reduce across ranks and store result on one rank (aggregation).
- **AllGather:** gather shards so each rank has full data (common in unshard steps, e.g., FSDP).
- **ReduceScatter:** reduce + scatter so each rank keeps only a shard (common for sharded gradients).

Gradient-sync reminder: In data parallel training, each GPU computes partial gradients on its mini-batch, then synchronization collectives are used before weight update.

5.1 AllReduce Algorithms

AllReduce combines tensors across GPUs and distributes reduced results to all ranks. It is used to synchronize gradients and keep model state consistent.

- Ring
- GPUs form a logical ring and exchange chunks with neighbors.
- Communication pattern is $2 \times (N - 1)$ steps.
- Tensor is split into chunks; reduce-scatter style pass plus all-gather pass.
- Pro: bandwidth-efficient, scales well for large messages.
- Con: latency accumulates with larger N ; weaker for small tensors.
- Interview cue: if one GPU is slower, the ring stalls (straggler effect).
- Tree
- Pair GPUs in a reduction tree, combine upward, then broadcast down.
- Complexity intuition: $O(\log N)$.

- Pro: lower latency, good for small messages.
- Con: less bandwidth-efficient; root/intermediate nodes can become hot spots.
- Prefer tree when latency dominates.
- Hierarchical
- Common in multi-node setups.
- Two-stage reduction:
 - intra-node reduction (e.g., NVLink domain)
 - inter-node reduction (e.g., InfiniBand/Ethernet fabric)
- Reduces cross-node traffic by exploiting faster intra-node links first.

5.2 InfiniBand + RDMA

- What is RDMA?
- Direct memory access across machines with minimal CPU/kernel involvement in the data path.
- Path intuition from your notes:
 - traditional: GPU $\rightarrow$ CPU/kernel $\rightarrow$ NIC $\rightarrow$ network $\rightarrow$ NIC/kernel $\rightarrow$ CPU $\rightarrow$ GPU
 - RDMA-style path reduces host-copy overhead
- How does EFA work?
- EFA = Elastic Fabric Adapter.
- Enables low-latency networking in cloud HPC/distributed workloads.
- Supports high-speed NCCL collective communication patterns in cloud settings.
- Why bypass CPU?
- CPU context switching, kernel interrupts, and extra copies increase latency and overhead.
- Latency vs bandwidth
- Large gradient tensors are often bandwidth-critical.
- Small control messages are latency-critical.
- Scaling behavior depends on both.

Troubleshooting Scenario

Scaling from 8 → 32 GPUs drops efficiency from 85% to 60%. Diagnose.

Diagnosis flow (from your notes):

1. Check GPU utilization first (`nvidia-smi`).
 - 90–100%: likely compute-bound
 - 40–60%: often communication or I/O bound
 - very low utilization: likely stalls
2. Profile timelines (`nsys`) and verify if NCCL dominates.
3. Use PyTorch profiler to measure NCCL time vs compute-kernel time.
4. Compute scaling efficiency and flag low values (your note: < 70% suspicious).

Network and communication checks:

- Enable NCCL debug logs:
 - `export NCCL_DEBUG=INFO`
 - `export NCCL_DEBUG_SUBSYS=ALL`
- Inspect InfiniBand/network tools: `ibstat`, `ibv_devinfo`, `sar -n DEV 1`, `iftop`, `nload`
- Check topology and PCIe/NVLink paths: `nvidia-smi topo -m`
- Watch for stragglers: one rank consistently slower/lower utilization
- Check CPU loader pressure (`htop`), NUMA placement, and thermal throttling (`nvidia-smi -q -d TEMPERATURE`)

Batch-size effect from your notes:

$$\text{per-GPU batch} = \frac{\text{global batch}}{\text{\#GPUs}}$$

If GPU count grows while global batch is fixed, per-GPU batch shrinks and communication can dominate. Typical fix in your notes: increase global batch and/or use gradient accumulation.

Quick reference flow:

- check GPU utilization
- profile NCCL time
- enable NCCL logs
- check network saturation

- check per-GPU batch size
- look for stragglers
- confirm hierarchical reduction behavior

Strong answer cues for “improve 32-GPU scaling”:

- increase batch size
- tune gradient bucket size
- enable/tune hierarchical AllReduce path
- reduce synchronization frequency
- improve network

6 GPU Architecture + Performance

Performance Reminder

$$\text{TFLOPs} \propto \text{cores} \times \text{clock} \times \text{FLOPs/cycle}$$

Performance is strongly affected by memory bandwidth, available TFLOPs, and Tensor Core utilization.

6.1 SM Architecture

SM Quick Recall

- **Warp:** 32 threads per warp (SIMT style).
- **Warp scheduler:** picks ready warps to hide latency.
- **Dispatcher:** issues instructions to pipelines/cores.
- If one warp stalls on memory, the scheduler switches to another warp.
- If threads in one warp diverge on control flow, paths serialize and throughput drops.
- **Occupancy:** active warps / max possible warps per SM.
- Achieved occupancy can differ from theoretical occupancy due to scheduling/resource limits.

- **Shared memory:** enables intra-block reuse and reduced global-memory traffic.
- **Registers:** fastest storage tier for thread-local values.

Library/tool notes from your content:

<i>Tool</i>	<i>Purpose</i>
cuBLAS	GPU-accelerated linear algebra
cuFFT	FFT library
cuDNN	Deep-learning kernels
cuda-gdb	Kernel debugging (breakpoints/step)
cuda-memcheck	Memory correctness checks

Block-size tradeoff from your notes:

<i>Choice</i>	<i>Tradeoff</i>
Larger blocks	Better shared-memory reuse and fewer redundant global loads; can lower occupancy.
Smaller blocks	Easier latency hiding and scheduling flexibility; less intra-block reuse and potentially lower arithmetic intensity.

6.2 Bottleneck Types

<i>Type</i>	<i>Symptoms</i>	<i>Typical fixes</i>
Compute-bound	High SM activity, memory bandwidth not maxed, kernels dominate runtime.	Improve Tensor Core use, fuse kernels, improve occupancy, reduce divergence.
Memory-bound	Bandwidth near peak, moderate SM activity, many stalls.	Tiling, shared-memory reuse, coalescing, fewer global reads/writes.
IO-bound	GPU idle, DataLoader maxed, disk/network busy.	Dataset caching, offline preprocessing, parallel data loading, faster storage (SSD/NVMe), reduced contention, distributed sharding with node-local caching.
Communication bound	Placeholder retained.	Placeholder retained.

Important CUDA terms:

- `gridDim.x`, `gridDim.y`: grid dimensions (number of blocks)
- `blockDim.x`, `blockDim.y`: threads per block dimensions
- `threadIdx.x`, `threadIdx.y`: thread coordinates inside block
- `blockIdx.x`, `blockIdx.y`: block coordinates inside grid

2D indexing reminder (matmul/conv-style):

$i = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y}$, $j = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$

Prompt

GPU utilization is 40%. How do you debug?

1. Confirm low utilization with `watch -n 1 nvidia-smi`.
2. Check detailed counters: `nvidia-smi dmon -s u`.
3. Check CPU loader bottlenecks: `htop`.
4. Check process/thread CPU placement: `ps -o pid,psr,pcpu,comm -p <PID>`.
5. Check data-loader I/O: `iostat -xm 1`.
6. Check filesystem/network I/O: `df -h, lfs df -h, sar -n DEV 1`.
7. Check communication dominance:
 - `export NCCL_DEBUG=INFO`
 - `export NCCL_DEBUG_SUBSYS=ALL`
8. Profile with Nsight Systems:
 - `nsys profile -o profile_report python train.py`
 - look for large NCCL blocks and GPU idle gaps
9. Check per-GPU batch size: $\text{per-GPU batch} = \text{global batch} / N$.
10. Check stragglers and thermals: `nvidia-smi, nvidia-smi -q -d TEMPERATURE`.
11. Check NUMA/topology and PCIe paths: `nvidia-smi topo -m`.
12. If kernels are too short/small, increase effective work per step.

7 Data Pipeline Scaling

Topics

- Sharding datasets
- DataLoader `num_workers`
- Prefetching
- Caching

- Metadata server overload

Pipeline details from your notes:

- ***DistributedSampler***: avoids duplicate shard reads across GPUs, improves balance across ranks.
- ***DataLoader workers***: each worker is a subprocess loading data in parallel.
- ***Prefetching***: loads the next batch while the GPU processes the current batch.
- Important flags from your notes: `prefetch_factor`, `persistent_workers`, `pin_memory`.
- `pin_memory` allocates page-locked host memory to speed CPU-to-GPU transfers.
- ***Caching***: cache dataset in RAM, cache preprocessed tensors, or use local NVMe instead of network FS.

<i>Issue</i>	<i>Symptoms</i>	<i>Fixes from your notes</i>
DataLoader starvation	Low GPU utilization; CPU and I/O high; profiling shows gaps between batches.	Increase workers, enable <code>pin_memory</code> , cache locally, increase batch size, use faster storage, use async preprocessing.
Metadata server overload	Random stalls, high IOPS, inconsistent speed, high %util/wait in <code>iostat -xm 1</code> .	Use tar/record files, prepack dataset, cache locally, increase striping.
Distributed amplification	Multiple processes load data simultaneously, amplifying I/O pressure and metadata contention.	Shard data, reduce small files, cache node-local.
Slow rank stall	Collectives (for example AllReduce) require all ranks; slowest rank sets step time.	Fix stragglers and rebalance pipeline.

Prompt

Why might training stall randomly in distributed jobs?

1. DataLoader starvation.
2. Metadata server overload.
3. Network file system congestion.
4. Straggler node.
5. Random garbage collection or memory spikes.
6. Background system activity (CPU, disk, network).

7.1 KV Cache for Autoregressive Inference

KV Cache Mental Model

In decoder-only inference, the model generates one token at a time. For each new token, you only need a *new query* and the historical *cached keys/values* from prior tokens.

From your notes, the key idea is to avoid recomputing old attention state repeatedly:

- Transformer layers produce Q, K, V per token.
- For the newest token, attention uses that token's Q_t against all previous $K_{1:t}$ and $V_{1:t}$.
- Recomputing K, V for old tokens every step is redundant.

KV-cache decode loop (cleaned from your notes):

1. Compute Q_t, K_t, V_t for the newest token only.
2. Append K_t, V_t to per-layer cache.
3. Read cached $K_{1:t}, V_{1:t}$.
4. Compute attention with Q_t and cached history.

Important Clarification

KV caching removes repeated *projection* work for past tokens, but attention over context is still sequence-length dependent. So decode compute does not become constant-time; it becomes much more efficient than full recomputation.

TTFT vs steady decode (from your notes):

- ***Prefill phase:*** process full prompt, build cache, highest initial compute cost.
- ***Decode phase:*** one-token steps using warm cache.
- Longer prompts increase prefill cost, so ***time-to-first-token (TTFT)*** rises.

<i>Phase</i>	<i>Main work</i>	<i>Typical bottleneck</i>
Prefill	Full prompt forward pass + KV cache build	Compute + memory bandwidth
Decode	Per-token forward with cached history	Memory bandwidth + cache capacity

Memory tradeoff (your point, organized):

- KV cache trades compute for memory.
- Cache grows with layer count, context length, and concurrent requests.
- For long contexts and many concurrent users, KV memory can exceed model weights.
- This motivates ***MQA/GQA*** to reduce KV-head memory pressure.

8 Fault Tolerance + Checkpointing

How it is done in PyTorch (from your notes): use DistributedSampler.

Listing 3: DistributedSampler sketch from notes

```
1 sampler = torch.utils.data.DistributedSampler(  
2     dataset,  
3     num_replicas=world_size,  
4     rank=rank,  
5     shuffle=True,  
6 )  
7 loader = DataLoader(dataset, sampler=sampler)
```

Why this matters (from your notes):

- avoids scenarios where GPUs read the same data shard
- avoids load imbalance across ranks
- reduces straggler effects from uneven sample counts

If dataset size is not divisible by world size:

- some ranks may process fewer samples, causing sync delays
- using `drop_last=True` can help keep step balance

DataLoader num_workers notes:

- each worker is a subprocess loading data in parallel
- too low: GPU waits for data (low utilization)
- too high: CPU thrashing, memory pressure, context-switch overhead
- starting point from your notes: around 4–8 workers per GPU (then tune by CPU/core budget)
- monitor with `htop`; increase until GPU utilization stops improving or CPU saturates

Prefetching + Caching Notes

- Prefetching loads the next batch while the GPU processes the current batch.
- Without prefetching, GPU idle gaps appear between batches.
- Shared filesystem failures from your notes: metadata overload, thousands of small file opens, random stalls.
- Small files hurt because each access requires metadata lookup and adds latency.
- Metadata overload (Lustre/NFS): random stalls, high IOPS, inconsistent speed.

- Detection: `iostat -xm 1` (high utilization and high wait time).
- Fixes from your notes: tar/record files, prepack dataset, cache locally, increase striping.

Possible questions from your notes:

<i>Question</i>	<i>Answer from your notes (cleaned)</i>
How do you detect DataLoader bottleneck?	GPU util low, CPU usage high, I/O high, profiling shows gaps between batches.
How would you improve data pipeline throughput?	Increase workers, enable pin memory, cache locally, increase batch size, use faster storage, use async preprocessing.
Why does distributed training amplify data bottlenecks?	Multiple processes load data simultaneously, increasing I/O pressure and meta-data contention.
Why does one slow rank stall everything?	Collective operations like AllReduce require all ranks; the slowest process determines global step time.

8.1 Checkpoint and Resume State

- *Sharded checkpoints*
- *Partial node failure*
- *Resume logic*
- *Optimizer state restoration*
- *RNG state*

Checkpointing details from your notes:

- Checkpointing can cause memory spikes if consolidation gathers full parameters temporarily.
- Reduce checkpoint time with sharded saves, fewer large files, less frequent saves, and async writing.
- For every parameter, optimizer stores momentum and variance; optimizer memory can be around $2\times$ model size.
- Also save step count and learning-rate scheduler state.
- Restoring only weights (without optimizer state) loses optimizer history and can behave like partial reset.

What to save (minimum from your notes):

- model weights (sharded)
- optimizer state (sharded)
- global step
- LR scheduler state
- RNG state
- ideally sampler state

RNG APIs in your notes:

- `torch.get_rng_state()`
- `torch.cuda.get_rng_state_all()`
- `numpy.random.get_state()`
- `random.getstate()`

Why this matters: dropout and data shuffling depend on RNG, so resume is not deterministic without restoring RNG state.

Hard Question

How do you checkpoint a 70B model efficiently?

- do not gather full model to one GPU
- save sharded checkpoints
- save optimizer state
- save training step + RNG state
- write fewer large files, not thousands of small files

Scale note from your content:

- in BF16, weights alone can be very large (your note: about 140 GB)
- Adam adds momentum and variance states, so total distributed training state can be much larger (your note: around 400 GB)
- gathering to one GPU can lead to OOM

What to implement from your notes:

- in FSDP or ZeRO-3, each GPU saves its parameter shard and optimizer shard
- use sharded checkpointing
- avoid consolidation unless exporting for inference

9 Mixed Precision + Numerical Stability

9.1 FP16 vs BF16

<i>Format</i>	<i>Bit layout from your notes</i>	<i>Key point</i>
FP16	16 bits total, 5-bit exponent, 10-bit mantissa	Smaller dynamic range; more prone to underflow/overflow.
BF16	16 bits total, 8-bit exponent, 7-bit mantissa	FP32-like exponent range; more stable in practice.

- Why BF16 is more stable: same exponent range as FP32, so large and small numbers are represented more safely.
- Why FP16 may diverge: gradients can underflow to zero or overflow to infinity due to limited exponent range.

9.2 Loss Scaling

- Needed because FP16 has a small dynamic range.
- Very small gradients may underflow to zero.
- Scale loss before backward, then divide gradients by the scale after backward.
- **Static scaling:** constant scale.
- **Dynamic scaling:** preferred in your notes; adjusts scale based on overflow detection.

9.3 Gradient Overflow

- Overflow happens when gradient values exceed representable range.
- Symptoms: loss becomes NaN, gradients contain `inf`, training diverges suddenly.
- Detection from your notes:
 - `torch.autograd.detect_anomaly()`
 - GradScaler checks
 - `torch.isnan(tensor).any()`
- Causes from your notes: large activations, unstable learning rates, insufficient loss scaling in FP16.
- Debug from your notes: check overflow, reduce learning rate, enable gradient clipping, switch to BF16, inspect layer outputs.

9.4 Stability Tradeoffs

- **Memory savings:** 16-bit instead of 32-bit gives roughly $2\times$ memory savings.
- **Throughput increase:** Tensor Cores are optimized for FP16/BF16.
- **Numerical risk:** reduced precision can increase rounding and accumulation error.

Typical strategy from your notes:

- store master weights in FP32
- compute in FP16 or BF16
- reduce in FP16 or BF16
- update in FP32

Common Q&A from your notes:

- Why keep master weights in FP32? To preserve update precision and reduce accumulated rounding error.
- When avoid mixed precision? Unstable models, small-model debugging, or unsupported hardware.
- Why can large batching increase instability risk? Larger batches can increase gradient magnitude and overflow risk at low precision.

10 Scheduler + Orchestration

10.1 Slurm

Slurm stands for *Simple Linux Utility for Resource Management*.

- `sbatch` vs `srun`
 - **sbatch:** non-interactive, queued, runs when resources are available; commonly used for long training jobs, production runs, and multi-node jobs.
 - **srun:** launches tasks; interactive or used inside allocation; common for launching distributed training processes.
- Partitions
 - Logical queues (for example: `gpu`, `cpu`, `highmem`, `debug`).

- Configured in `slurm.conf`.
- Each partition has time limits, node restrictions, and resource limits.
- Used to separate workloads, enforce policies, and control access for different job types.
- GPU allocation
 - In Slurm: `#SBATCH -gres=gpu:4`
 - GRES = generic resources.
 - Slurm assigns GPU IDs and sets `CUDA_VISIBLE_DEVICES`.
 - Multi-node example: `#SBATCH -nodes=2`
`#SBATCH -ntasks-per-node=4`
`#SBATCH -gres=gpu:4`
- Fairshare
 - Slurm tracks historical usage.
 - If you used many GPUs recently, your priority can drop.
 - Fairshare improves fairness and prevents monopolization.
 - Priority factors: fairshare, job age, QoS, partition priority.

10.2 Kubernetes

Here scheduling is for containers, not raw processes. Typical use from your notes: cloud GPU clusters, microservices, ML platforms (EKS/GKE/AKS).

- Pods
 - Smallest deployable unit.
 - Contains one or more containers with shared network and storage.

Listing 4: Pod GPU limit sketch from notes

```
1 resources:
2   limits:
3     nvidia.com/gpu: 1
```

- GPU resource requests
 - Kubernetes uses a device plugin for GPU resources.

Listing 5: Kubernetes GPU request/limit sketch from notes

```
1 resources:
2   requests:
```

```

3     nvidia.com/gpu: 2
4     limits:
5     nvidia.com/gpu: 2

```

- Scheduler assigns a node with enough GPUs and exposes them to the container.
- Kubernetes knows GPU resources via device-plugin registration.
- Scheduling policies
 - Scheduler considers CPU, memory, GPU, node labels, taints/tolerations, and affinity.

Listing 6: Node selector sketch from notes

```

1 nodeSelector:
2   gpu-type: a100

```

- Auto-scaling
 - Kubernetes can scale pods and clusters.
 - Useful for inference and dynamic training environments.

Q&A from your notes:

<i>Question</i>	<i>Answer from your notes (cleaned)</i>
Why is autoscaling useful for ML?	Inference demand fluctuates, and training clusters may need to add/remove GPU nodes dynamically.
Slurm vs Kubernetes for ML workloads?	Slurm: HPC-oriented, better for tightly coupled multi-node training, native MPI/NCCL integration, lower overhead. Kubernetes: container-native, easier cloud integration, built-in autoscaling, flexible workload types.
Why might distributed training hang under Slurm?	Wrong environment variables, firewall between nodes, incorrect tasks-per-node, misconfigured NCCL.
How does Slurm integrate with DDP?	Slurm sets env vars like <code>SLURM_PROCID</code> and <code>SLURM_NODEID</code> that can initialize rank and world size.
Why might Kubernetes be worse for tightly coupled training?	Higher networking overhead, container abstraction layers, and less optimized HPC fabric integration compared to Slurm.
How would you schedule multi-node DDP on Kubernetes?	Use StatefulSets or custom job controllers with proper rank assignment and headless services for communication.
What is gang scheduling?	Allocate all resources for a distributed job simultaneously to avoid partial scheduling.

11 Storage Systems

Storage Layers from your notes

- local NVMe (fast per node)
- parallel filesystem (Lustre/GPFS/BeeGFS)
- possible object storage (S3-like)

For large-scale ML, shared parallel storage is critical.

- Lustre architecture Lustre is a parallel distributed filesystem used in HPC. It separates metadata and data storage.

Components:

1. **MDS (metadata server)**: stores filenames, permissions, directory structure, and file-location mapping; handles `open()`, `close()`, `stat()` metadata operations.
2. **MDT (metadata target)**: storage device behind MDS.
3. **OSS (object storage server)**: handles actual file data.
4. **OST (object storage target)**: stores file data blocks.

Files are striped across multiple OSTs for parallel access.

What happens when you open a file?

- client asks MDS for metadata
- MDS returns file layout
- client reads data directly from OST

Q: Why separate metadata and data? To allow parallel data access across multiple storage targets while centralizing lighter metadata operations.

Q: What happens when metadata server is overloaded? File open/stat operations slow down, causing random stalls even when data bandwidth is fine.

- Metadata server This is where many ML workloads fail.

ML datasets often contain:

- millions of small image files
- many file opens per batch
- multiple DataLoader workers per GPU

Each worker may call `open()` thousands of times. If many GPUs do this simultaneously, metadata storms occur.

Symptoms:

- random training stalls
- high IOPS but low throughput
- GPU utilization fluctuates
- CPU waits on I/O

Detection:

- `iostat -xm 1` (look for high %util and high wait)
- `lfs df -h` on Lustre

Q: Why do small files hurt parallel filesystems? Each file access needs metadata lookup, which can overwhelm the metadata server.

Q: Why might training stall randomly? Metadata server bottlenecks under high concurrent file-open rates.

- Small file problem Very common in ML.

Example from your notes: ImageNet has many small JPEG files.

Per batch, many files are opened repeatedly across many GPUs.

Problem:

- high metadata overhead
- poor sequential read efficiency
- high open/close syscall overhead

Solutions from your notes:

- pack into larger containers: TFRecord, WebDataset, LMDB, HDF5
- increase striping: `lfs setstripe -c 4 myfile`
- cache locally (for example: `rsync -av dataset /local_nvme/`)

Q: How to fix the small-file problem? Batch files into larger shard files to reduce metadata lookups and improve sequential I/O.

- Parallel IO Lustre allows files to be striped across multiple OSTs so clients can read in parallel.

Striping improves bandwidth; `lfs getstripe filename` shows file striping.

Why parallel I/O matters:

- large model checkpointing (100+ GB)
- many GPUs reading simultaneously

- need aggregate bandwidth

Without striping:

- single-OST bottleneck
- slower checkpoint save/load

What is file striping? Dividing file data across multiple storage targets to enable parallel read/write.

When would you increase stripe count? For large files like checkpoints to increase aggregate throughput.

Common GPU cluster storage failures from your notes:

- GPUs at 40% utilization caused by data pipeline I/O bottlenecks and metadata overload
- checkpoints taking minutes due to single-OST bottleneck, insufficient striping, or network congestion
- random stalls due to shared filesystem contention and too many simultaneous jobs

Q: Why might local NVMe be faster than Lustre? It removes network overhead and metadata contention, giving lower latency and more consistent per-node throughput.

Q: Why is a parallel filesystem needed? Multi-node training needs shared access to datasets and checkpoints.

Q: Why can distributed training amplify I/O issues? Many ranks read simultaneously, multiplying metadata and bandwidth pressure.

Q: Why does checkpointing stall other jobs? Large parallel writes can saturate shared OST bandwidth and metadata operations.

Storage and GPU scaling connection: scaling GPU count increases data-read parallelism, checkpoint write pressure, and metadata operations, so storage becomes a bottleneck at scale.

12 Cloud vs On-Prem Tradeoffs

- Spot instances Cloud providers (for example AWS/GCP) offer discounted compute capacity with interruption risk.

Your notes: often around 50–90% cheaper; useful for hyperparameter sweeps, non-critical jobs, and fault-tolerant workloads.

Risks: instance termination, loss of local storage, and distributed job collapse.

Use spot safely (from your notes): frequent checkpointing, TorchElastic, distributed job managers, persistent object storage for checkpoints.

Q: How would you train a 70B model on spot instances? Only if the framework supports fault tolerance and elastic recovery; otherwise interruption risk may waste compute.

Q: How make distributed training resilient to spot termination? Use elastic training, periodic checkpointing to persistent storage, and automatic rank rejoin/restart.

- Cost optimization On-prem vs cloud cost model from your notes includes GPU hours, storage, bandwidth, data egress fees, idle resources, and networking.

Q: Why might cloud training be more expensive? High GPU-hour rates plus large-scale distributed networking/storage costs.

Q: When is on-prem cheaper? When GPU utilization is consistently high over long periods and hardware cost is amortized.

Q: Hidden cloud costs in ML? Data egress fees and underutilized reserved capacity.

- Elastic scaling Ability to add/remove nodes dynamically and adjust cluster size with demand.

Cloud strengths: autoscaling groups, Kubernetes cluster autoscaler, dynamic node provisioning.

On-prem limitation: fixed node pool; cannot instantly add GPUs; needs hardware procurement.

TorchElastic points from your notes:

- allows ranks to leave/join
- reforms process groups
- handles failures

Scenario from your notes: training can lose 1–2 workers, restart, and continue from latest checkpoint.

Instead of fixed world size, TorchElastic uses rendezvous coordination, dynamic membership (min/max workers), and automatic restart on failure.

When a worker dies: collectives fail, worker group restarts, ranks/world size reform, and script resumes from checkpoint.

Q: Why is elastic scaling hard for distributed training? Most frameworks assume fixed world size and static ranks; changing size requires communication-group changes.

Q: Why is elasticity easier for inference than training? Inference is stateless per request, while training requires synchronized gradient updates.

Listing 7: TorchElastic resume sketch from notes

```
1 import os
2 import time
3 import torch
```

```

4 import torch.distributed as dist
5 from torch.nn.parallel import DistributedDataParallel as DDP
6
7 CHECKPOINT_PATH = "ckpt.pt"
8
9 def setup_distributed():
10     """
11     TorchElastic (torchrun) sets these env vars:
12     - RANK, LOCAL_RANK, WORLD_SIZE
13     - MASTER_ADDR, MASTER_PORT (or rendezvous handles it)
14     """
15     dist.init_process_group(backend="nccl")
16     local_rank = int(os.environ["LOCAL_RANK"])
17     torch.cuda.set_device(local_rank)
18     return local_rank
19
20 def build_model():
21     model = torch.nn.Linear(1024, 1024).cuda()
22     return model
23
24 def save_checkpoint(ddd_model, optimizer, step: int):
25     if dist.get_rank() == 0:
26         ckpt = {
27             "step": step,
28             "model": ddp_model.module.state_dict(),
29             "optim": optimizer.state_dict(),
30         }
31         torch.save(ckpt, CHECKPOINT_PATH)
32
33 def load_checkpoint(ddd_model, optimizer):
34     if os.path.exists(CHECKPOINT_PATH):
35         ckpt = torch.load(CHECKPOINT_PATH, map_location="cpu")
36         ddp_model.module.load_state_dict(ckpt["model"])
37         optimizer.load_state_dict(ckpt["optim"])
38         return int(ckpt["step"])
39     return 0
40
41 def main():
42     local_rank = setup_distributed()
43     model = build_model()
44     ddp_model = DDP(model, device_ids=[local_rank])
45     optimizer = torch.optim.AdamW(ddp_model.parameters(), lr=1e-3)
46     start_step = load_checkpoint(ddp_model, optimizer)

```

```

47
48     for step in range(start_step, start_step + 10_000):
49         optimizer.zero_grad(set_to_none=True)
50         x = torch.randn(32, 1024, device="cuda")
51         y = ddp_model(x).sum()
52         y.backward()
53         optimizer.step()
54         if step % 200 == 0:
55             save_checkpoint(ddp_model, optimizer, step)
56         if dist.get_rank() == 0 and step % 200 == 0:
57             print(f"[step={step}] world_size={dist.
58                 get_world_size()} ok")
59
58     dist.destroy_process_group()
59
60
61 if __name__ == "__main__":
62     main()

```

Here is the Slurm script to run this:

Listing 8: Slurm + torchrun launch sketch from notes

```

1  # inside an allocation with 2 nodes, 4 GPUs/node = 8 procs total
2  export RDZV_ENDPOINT="$(scontrol show hostnames $SLURM_NODELIST
3      | head -n1):29400"
4
5  export NNODES=$SLURM_NNODES
6
7  srun bash -c '
8      torchrun \
9          --nnodes='"$NNODES"' \
10         --nproc_per_node=4 \
11         --rdzv_backend=c10d \
12         --rdzv_endpoint='"$RDZV_ENDPOINT"' \
13         --rdzv_id=job123 \
14         --max_restarts=10 \
15         train_elastic.py

```

RDZV endpoint is where workers meet, `max_restarts` controls tolerated failures, and `rdzv_id` is the rendezvous room name that must match.

Q: If a worker dies, does training continue instantly? Not usually. Current process group breaks; TorchElastic restarts workers and reforms ranks; resume happens via checkpoint load.

Q: What do you need to checkpoint for correct resume? Minimum: model weights, optimizer state, global step. Ideally: RNG state and sampler state.

Q: Does world size change automatically? It can, depending on min/max config; many setups keep world size constant and only restart on failures.

Biggest pitfall from your notes: script restarts from scratch unless resume-from-checkpoint logic is implemented.

With autoscaling, a job can start at minimum nodes and grow toward max nodes as more capacity arrives.

- Network topology differences Uses AWS Elastic Fabric Adapter (EFA) instead of InfiniBand in many cloud setups.

Data is often in S3, but large multi-node training is usually better when data is staged locally or served by a high-performance filesystem to avoid high latency.

Uses Scalable Reliable Datagram (SRD), designed to handle congestion via multipath networking.

Large-scale training depends on AllReduce bandwidth, low latency, and network determinism; cloud can introduce higher jitter and bandwidth variability.

Jitter is variability in network latency. It causes synchronization delays and amplifies straggler effects in distributed training.

13 Researcher Productivity Metrics

<i>Metric</i>	<i>What to measure from your notes</i>	<i>Interpretation from your notes</i>
GPU utilization	<code>nvidia-smi</code> , <code>nvidia-smi dmon</code> , <code>nsys</code> .	Under 40% often underperforming; around 90% can indicate healthy compute-bound behavior (but still check queue/convergence).
Queue time	submission $\rightarrow$ job start; track average and P95 wait time.	Lower queue time improves researcher iteration speed.
Time-to-convergence	wall-clock time to target accuracy/loss (not epochs).	Most important end metric; throughput alone is insufficient.
Experiment reproducibility	controlled RNG, stable environment, deterministic ops, proper checkpointing.	Without reproducibility, debugging and conclusions are unreliable.
Cost per training run	cost = GPU hours $\times$ cost per GPU hour + storage/network/engineering.	Key for cost per experiment/model/inference impact.

Notes from your content:

- Low utilization causes: DataLoader bottleneck, network bottleneck, small batch size, poor overlap, CPU starvation.
- Reduce queue time with scheduling policy improvements, fairshare tuning, dedicated partitions, preemption, and cloud elasticity.

- If throughput improves but time-to-convergence does not: check scaling efficiency, communication bottlenecks, LR scaling, and batch-size optimization effects.
- Improve reproducibility: fix seeds, log environment, version data/code, save full training state.
- Reduce run cost: increase scaling efficiency, use mixed precision, optimize batch size, reduce failed runs, use spot safely.

Prompt

How do you measure infrastructure impact?

1. Define baseline:

- GPU utilization
- queue time
- step time
- time to convergence
- failure rate
- cost per run

2. Apply infrastructure improvements:

- better networking
- improved data loader
- FSDP adoption
- storage caching

3. Remeasure and compare:

- percent improvement in step time
- reduction in queue time
- reduction in job failures
- reduction in cost per convergence

14 Rapid-Fire Questions Section

Rapid-Fire: One-Line Answer Bank

- ***Why doesn't scaling linearly improve speed?*** Distributed training adds communication and synchronization overhead (for example, AllReduce). As GPU count grows, network contention, latency amplification, and straggler effects grow too. If global batch size stays constant, per-GPU compute shrinks and communication-to-compute ratio worsens, so scaling becomes network-bound.
- ***What is gradient accumulation?*** Gradients from multiple forward/backward micro-steps are accumulated before one optimizer step. This simulates a larger effective batch without requiring a larger per-step memory footprint.
- ***What is microbatching?*** A large batch is split into smaller microbatches processed sequentially (often in pipeline parallelism). This keeps devices busier and reduces pipeline idle time, but increases scheduling complexity.
- ***What causes NCCL deadlocks?*** Mismatched collective calls across ranks, crashed ranks, wrong world-size/rank initialization, firewall/network misconfiguration, or version/runtime mismatch. One misbehaving rank can stall the job.
- ***Why do small files hurt Lustre?*** Each file open requires metadata operations. With many workers and many GPUs, metadata requests explode and overload the metadata server, causing random stalls and low GPU utilization.
- ***Why does large batch size hurt generalization?*** Large batches reduce gradient noise and can converge to sharper minima that generalize worse. They also require correct learning-rate scaling.
- ***What is straggler effect?*** One slow worker delays all others at synchronization points, so overall step time is determined by the slowest rank.
- ***What is NUMA?*** Non-uniform memory access: memory latency depends on CPU socket locality. Remote-memory/PCIe access paths can hurt performance.
- ***What is PCIe bottleneck?*** Host-device or inter-device transfers saturate PCIe bandwidth, limiting throughput when communication/memory movement dominates.
- ***What is kernel fusion?*** Multiple operations are combined into one kernel launch to reduce memory traffic and launch overhead, improving locality and throughput.
- ***What is compute intensity?*** Arithmetic operations per byte transferred (FLOPs/byte). High intensity is usually compute-bound; low intensity is usually memory-bound.

- ***Why does communication dominate at scale?*** As rank count rises, collective overhead grows while per-rank compute can shrink, so synchronization time can dominate step time.
- ***What happens if one rank crashes?*** In synchronous training, collectives fail/hang and the job typically stops. In elastic setups, the worker group restarts and resumes from checkpoint.
- ***What is asynchronous training?*** Workers update parameters without strict step-by-step synchronization. Throughput can increase, but gradient staleness may hurt convergence stability.
- ***What is pipeline bubble?*** Idle time at pipeline fill/drain phases in pipeline parallel training. Bubble overhead increases when stage count is high relative to microbatch count.
- ***What is memory fragmentation?*** Repeated allocations/deallocations leave non-contiguous free memory blocks; OOM can occur despite enough total free memory.
- ***What is KV cache?*** During autoregressive inference, key/value tensors from previous tokens are stored and reused to avoid recomputing full attention each step.
- ***What is tensor parallel shard?*** A partition of large tensors/weights across GPUs (row-wise or column-wise). Each GPU computes partial outputs, then results are gathered/reduced.
- ***What is gradient clipping?*** Limits gradient norm/magnitude to prevent exploding gradients and improve training stability.
- ***What is elastic training?*** Training that tolerates worker failures/membership changes via rendezvous and restart logic, typically resuming from checkpoints.

15 Whiteboard System Design Prompts

Design Prompts

- Design multi-tenant GPU scheduling system
- Design distributed LLM inference cluster
- Design hyperparameter search system

- Design fault-tolerant distributed training
- Design GPU utilization monitoring dashboard

15.1 Design multi-tenant GPU scheduling system

Clarify first: on-prem vs cloud vs hybrid.

Core design from your notes:

1. Fairness and quotas: guaranteed quota + burst quota per team; weighted fairshare/DRF-style policy to prevent monopolization.
2. Scheduling strategy: two-level scheduling (choose tenant, then highest-priority job in tenant). Support interactive, production inference, and batch classes.
3. Placement strategy: topology-aware placement; prefer same NVLink domain/same node before cross-node. Use gang scheduling for multi-node distributed jobs.
4. Preemption and isolation: preempt best-effort jobs with checkpoint grace periods; enforce GPU/network/storage isolation per tenant.
5. Observability: expose clear pending reasons (quota exceeded, waiting for gang allocation, topology constraints).

Questions to expect:

- How do you prevent fragmentation? Bin-pack small jobs and reserve capacity pools for 8–16 GPU gang allocations.
- How do you handle distributed jobs? Gang scheduling so all required GPUs are allocated atomically.
- How do you keep utilization high? Backfilling + burst capacity when quotas are under-used.
- Hardest problem? Balancing fairness and utilization while avoiding fragmentation and starvation.

15.2 Design distributed LLM inference cluster

Clarify first: latency target (P50/P99), tokens/sec, traffic shape, model sizes, hardware, and multi-tenancy constraints.

Architecture from your notes:

- LLM-aware router in front of GPU worker groups.
- Continuous batching + KV cache management.
- Admission control by tenant quotas and estimated KV-memory footprint.

- Route by cache locality and headroom.
- Use tensor parallelism for larger models; combine TP within node + pipeline across nodes when needed.
- Reliability via health checks, circuit breakers, graceful draining, and canary rollouts.
- Observability: TTFT, per-token latency, tokens/sec, KV eviction/swap rates.

Questions to expect:

- Main bottleneck? Often memory bandwidth + KV cache capacity/management.
- How keep P95 low with batching? Separate interactive pools, cap token budgets, prioritize short requests.
- What if KV cache does not fit? Paged KV cache and controlled CPU/NVMe offload fallback.
- How do you scale? Add model-group replicas and autoscale per model and priority tier.

15.3 Design hyperparameter search system

Clarify first: objective (best quality, fastest time-to-best, or cost cap), workload type, search space, cluster constraints, and reporting metrics.

Core architecture from your notes:

- Orchestrator proposes trials.
- Scheduler launches trials (Slurm/Kubernetes) with quotas and fairshare.
- Workers report metrics and checkpoints.
- Default algorithm: ASHA/Hyperband for scalable early stopping; optional TPE for expensive trials.
- Intermediate metrics stream back for early stop and reprioritization.
- Checkpoint/resume is first-class for preemption/spot.
- Centralized stores for trial configs, metrics, artifacts, reproducibility metadata, and UI ranking.

Questions to expect:

- Why random search over grid? In high dimensions, random exploration is often more efficient.
- Why ASHA? Massively parallel and saves GPU time by early-stopping bad trials.
- How handle noisy metrics? Smoothing, minimum-step rules, repeated runs for top candidates.

- Biggest bottleneck? GPU availability/scheduling and storage metadata pressure if checkpointing too frequently.
- How avoid wasting GPUs? Early stopping, OOM/NaN detection, health checks, automatic retry/resume.

15.4 Design fault-tolerant distributed training

Design summary from your notes:

- Treat training as a restartable worker group.
- Use consistent sharded checkpoints.
- Add watchdog/heartbeat-based failure and hang detection.
- Use elastic rendezvous so workers can reform and resume from latest committed checkpoint.
- Save optimizer state + RNG state; avoid full-state gather that can cause OOM.

Questions to expect:

- What if one rank crashes in DDP? Collectives fail/hang; job restarts in elastic setup.
- How checkpoint FSDP/ZeRO-3 efficiently? Save per-rank shards; avoid full gather; include optimizer/scheduler/RNG.
- How avoid corrupted checkpoints? Two-phase commit (temp + manifest + atomic done marker).
- How detect hangs vs slow steps? Heartbeat/progress thresholds + correlate with NCCL logs and GPU utilization.
- Biggest pitfall? Not saving optimizer/RNG state; full checkpoint consolidation causing OOM; misdiagnosing DataLoader vs NCCL bottlenecks.

15.5 Design GPU utilization monitoring dashboard

Goal from your notes: Build a Grafana dashboard backed by Prometheus, enriched with Slurm/Kubernetes metadata and application training metrics.

Core dashboard views:

- SM utilization
- NCCL time
- DataLoader I/O time
- throttling/thermal signals

- tenant-level waste attribution

Questions to expect:

- How map GPU utilization to a specific job? Join scheduler allocation metadata (job → node → GPU IDs) with per-GPU metrics labels.
- Most actionable metrics? SM utilization + NCCL time + DataLoader time.
- What alerts? GPUs idle while allocated, ECC/Xid spikes, thermal throttling, NCCL failures, filesystem latency spikes, sustained low utilization by tenant.
- How avoid metric-overload cost? 1–5s scrape for critical GPU metrics, downsample long-term, and selectively enable high-cardinality labels.

15.6 Simulation 1: Multi-Tenant Heavy Transformer Training (Communication/Network Bottleneck)

Scenario from your notes

Scale from 8 GPUs on one NVLink node (~ 85% efficiency) to 32 GPUs across 4 InfiniBand-connected nodes, and efficiency drops to ~ 60% with high GPU idle time.

Diagnostic flow (cleaned):

1. Check coarse utilization first (`nvidia-smi`, DCGM): low SM util + high wait usually means comm or input bottleneck.
2. Use deterministic profiling (`nsys` + PyTorch profiler): quantify NCCL time vs compute time.
3. Inspect NCCL behavior:
 - `NCCL_DEBUG=INFO, NCCL_DEBUG_SUBSYS=ALL`
 - verify topology and fabric path (`nvidia-smi topo -m`)
 - confirm traffic is actually on InfiniBand/RDMA, not fallback paths

4. Check batch math:

$$\text{per-GPU batch} = \frac{\text{global batch}}{\text{\#GPUs}}$$

If global batch is fixed while GPU count increases, per-GPU compute shrinks and communication dominates.

Why Scaling Stops Being Linear

Synchronous training has barrier-like behavior at collectives. One slower rank (CPU jitter, loader delay, network tail latency) can stall all others, so end-to-end step time

tracks the slowest rank.

Mitigations from your notes (organized):

- Increase global batch (or use gradient accumulation) to improve compute/communication ratio.
- Tune DDP bucket size for overlap of backward compute and communication.
- Use hierarchical collectives for multi-node jobs (intra-node first, then inter-node).
- Keep TP-heavy communication mostly intra-node when possible.

<i>Transport</i>	<i>Path cost</i>	<i>Training impact</i>
TCP/IP sockets	More host copies + kernel/network stack overhead	Higher latency and CPU overhead at scale
RDMA (InfiniBand/EFA)	NIC offload + lower software overhead data path	Better collective latency/bandwidth behavior

15.7 Simulation 2: Memory Monster (100B model on 64 A100 GPUs)

Scenario from your notes

Design distributed training for a 100B-parameter model. The main problem is memory capacity, not raw compute throughput.

Why vanilla DDP fails (cleaned math from your notes):

- FP16 weights: 2 bytes/param $\Rightarrow$ 200 GB.
- FP16 gradients: 2 bytes/param $\Rightarrow$ 200 GB.
- FP32 master weights: 4 bytes/param $\Rightarrow$ 400 GB.
- Adam states (m, v): 8 bytes/param $\Rightarrow$ 800 GB.

Total model-state footprint $\approx$ 16 bytes/param $\Rightarrow \sim 1.6$ TB (excluding activation peaks).

Implication

No single 80 GB A100 can hold this full training state. You need sharding and model-parallel strategies, not plain replicated DDP.

Strategy stack from your notes:

- **TP**: shard large matrix multiplies (usually best intra-node on NVLink/NVSwitch).
- **PP**: split layers into stages; manage bubble overhead with microbatching.

- **ZeRO/FSDP:** shard optimizer, gradients, and parameters to reduce per-GPU steady-state memory.

ZeRO stage intuition (as you described):

- ZeRO-1: shard optimizer states.
- ZeRO-2: shard optimizer states + gradients.
- ZeRO-3/FSDP FULL_SHARD: shard optimizer states + gradients + parameters.

$$\text{Model-state memory per GPU (rough)} \propto \frac{1}{N}$$

for fully sharded states across N ranks, with increased communication.

Checkpoint failure mode from your notes:

- Training is stable, but periodic checkpoint save causes OOM.
- Root cause: accidental consolidation/all-gather of full model state onto one rank/device.

Fix (organized):

- Use distributed/sharded state dict checkpointing.
- Each rank writes its own parameter and optimizer shards.
- Save full resume context: model shards, optimizer shards, global step, LR scheduler, RNG state.

15.8 Simulation 3: Data Starvation + Metadata Storm

Scenario from your notes

Train a large vision model with hundreds of millions of small JPEG files. Observed: SM utilization around 40%, CPU near 100%, high I/O wait, low effective storage throughput.

Diagnosis (organized):

- Low InfiniBand utilization and high CPU/I/O wait suggest input pipeline bottleneck, not NCCL.
- Too many small-file `open/stat` operations overwhelm metadata services.
- DataLoader workers stall waiting for metadata/file opens, then GPU queues drain.

Why this happens on Lustre-like systems (from your notes):

- Metadata (MDS/MDT) and data (OSS/OST) are separate paths.
- Small files create heavy metadata traffic before data reads can begin.
- High fan-out from many nodes $\times$ many workers can create a metadata storm.

Important Clarification

`pin_memory=True` improves host-to-device transfer efficiency, but it does **not** solve metadata bottlenecks caused by millions of small files.

Triage steps from your notes:

1. Tune `num_workers` empirically (avoid over-saturating CPU and metadata path).
2. Enable `pin_memory=True` for faster/asynchronous H2D staging.
3. Use `prefetch_factor` and `persistent_workers=True`.
4. Repack small files into larger sequential shards (WebDataset/TFRecord/LMDB).
5. Add node-local caching on NVMe for repeatedly used datasets.

<i>Lever</i>	<i>What it fixes</i>	<i>What it does not fix</i>
<code>pin_memory=True</code>	H2D transfer overhead and overlap	Metadata-server overload
Increase <code>num_workers</code> (carefully)	CPU-side decoding/data prep parallelism	Small-file metadata storm at high worker counts
Shard dataset into large archives	Metadata lookup amplification + random small-file I/O	Poor model/network config issues
Node-local NVMe cache	Network/storage-array contention for reused data	First-load preprocessing cost

Orchestration choice after pipeline fixes (from your notes):

- ***Slurm strengths:*** topology-aware placement for tightly coupled synchronous training.
- ***Kubernetes strengths:*** elasticity and cloud-native operations.
- ***Kubernetes caveat for large distributed training:*** usually needs gang scheduling and HPC-network-aware extensions (device plugins, multi-NIC/CNI tuning, scheduler extensions).

16 Your Personal Deep Dive Stories

16.1 WaveHPC Impact Story

Situation: WaveHPC supported multi-GPU distributed training workloads across shared cluster resources. As adoption increased, you observed inconsistent GPU utilization, long queue times, and inefficient scheduling for multi-node jobs.

Task: Improve cluster efficiency and ensure fair, scalable execution for distributed training workloads across Slurm-based scheduling and Kubernetes-based services.

Action:

- Profiled cluster-wide GPU utilization with DGCM + Slurm metadata to identify idle GPUs during active allocations.
- Analyzed job logs and NCCL traces to detect communication bottlenecks and straggler effects.
- Identified workload imbalance across partitions and underutilized nodes.
- Implemented automation pipelines for:
 - job onboarding (correct `ntasks`, GPU binding, NCCL environment setup)
 - resource-aware Slurm submission defaults
 - monitoring dashboards for SM utilization, I/O wait, and network usage
- Tuned job packing and recommended per-GPU batch scaling to reduce communication overhead.
- Coordinated Slurm and Kubernetes CPU-pool segmentation for better multi-tenant isolation.

Result:

- improved cluster GPU utilization by **18%**
- reduced queue-time variance for multi-GPU jobs
- decreased idle-GPU time during active allocations
- established repeatable workload templates for distributed training

Follow-up questions to expect:

- What metric defined utilization? Primarily SM utilization, with memory utilization and network throughput for bottleneck context.
- How did you separate compute underutilization from communication stalls? Low SM + high network implies communication bound; low SM + high CPU/I/O implies input bound.
- How did you detect stragglers? Compared per-rank step times/utilization across ranks.
- What happens when one rank is slower? Other GPUs block at synchronization; scaling efficiency drops.
- How does small per-GPU batch hurt scaling? Less compute per step makes communication cost dominate.

- How would you debug 40% GPU utilization? Check SM/memory utilization, NCCL time, DataLoader bottlenecks, network saturation, and stragglers.
- Why were GPUs idle while allocated? Likely communication stalls, DataLoader bottlenecks, or incorrect GPU binding.
- How prevent one tenant from monopolizing GPUs? Quotas + fairshare scheduling.

16.2 High-Performance Matrix Inversion via Custom GPU Kernels

Situation: Matrix inversion on $5k \times 5k$ matrices was CPU-bound and became a bottleneck in numerical workloads, especially during repeated benchmarking.

Task: Design a GPU-based implementation that improves runtime while preserving numerical stability and correctness, and understand what happens inside the "black box" beyond cuBLAS.

Action:

- Profiled the CPU baseline and identified heavy memory movement, serialization in Gaussian-elimination phases, and poor cache reuse.
- Implemented block-based Gaussian elimination kernels in Triton and OpenCL for cross-vendor practice.
- Applied tiling to increase arithmetic intensity.
- Used shared/local memory to cache pivot rows and reduce global-memory pressure.
- Carefully synchronized pivot updates across blocks.
- Validated correctness with $A \times A^{-1} = I$ checks and randomized vector tests.

Result:

- achieved **1.5×** speedup
- reduced runtime by **<50%** vs optimized CPU baseline
- confirmed that memory layout and synchronization patterns dominated performance
- gained practical intuition for memory hierarchy effects on throughput

Follow-up questions to expect:

- Was it compute-bound or memory-bound? Mostly memory-bound for large matrices.
- How did you know? SM utilization was not maxed while memory throughput stayed high.
- Why tiling? Better cache reuse and fewer global-memory loads.

- What limited further speedup? Memory bandwidth and synchronization around pivot steps.
- What is memory coalescing? Threads in a warp access contiguous memory so loads combine efficiently.
- What is register pressure? Too many per-thread variables reduce occupancy and can cause spill to memory.

16.3 GPU-Accelerated Image Processing Pipeline

Situation: High-resolution image filters (for example blur and sharpen) were CPU-bound, causing preprocessing bottlenecks.

Task: Build a GPU-accelerated version with stable performance across different image sizes.

Action:

- Designed custom CUDA kernels in C++.
- Tuned thread-block dimensions to maximize occupancy.
- Used shared memory to cache image tiles.
- Ensured coalesced pixel access to reduce global-memory latency.
- Benchmarked against Python/PyTorch pipelines and validated correctness.
- Compared kernel launch configs to minimize divergence.

Result:

- achieved up to $8\times$ speedup vs CPU baseline
- maintained pixel-level numerical parity
- showed scaling with image resolution

Follow-up questions to expect:

- How did you choose block size? Empirical tuning (for example 16×16 and 32×32) to balance occupancy and shared-memory usage.
- What is warp divergence? Threads in the same warp take different branches, reducing efficiency.
- Was it memory-bound? Yes, pixel operations mainly stress memory bandwidth.
- Why $8\times$ speedup? Massive parallelism + shared-memory caching + coalesced loads.

16.4 Convolutional Layer Optimization with Triton

Situation: cuDNN hides convolution internals, making kernel-level bottlenecks hard to inspect directly.

Task: Implement and optimize a 2D convolution kernel to deepen understanding of GPU memory hierarchy and execution mapping.

Action:

- Built custom 2D convolution kernels in Triton.
- Experimented with tile sizes, block/thread mapping, shared-memory staging, and loop unrolling.
- Compared runtime against PyTorch built-in convolution.
- Analyzed memory-throughput vs arithmetic-intensity tradeoffs.

Result:

- achieved up to **1.5×** speedup on mid-size feature maps
- developed intuition for memory-bound vs compute-bound regimes
- understood how tiling affects cache reuse and where kernel fusion helps

Follow-up questions to expect:

- Why can convolution be memory-bound? Large feature maps drive high memory traffic relative to compute.
- How does tiling help? Increases data reuse in shared memory and reduces global-memory loads.
- What is kernel fusion? Combine multiple operations into one kernel to reduce memory traffic.